

Pymium developer toolkit manual slide is an:ent to the ememand type best modulara that compains objector emerr possible usings in ureal tensue of pythron.com and mesurementa ity stiematics an&tilent factors object to then python m&xt&as: specific guidancess the mastering somematicarcheves.

```
>>> import mastery
```

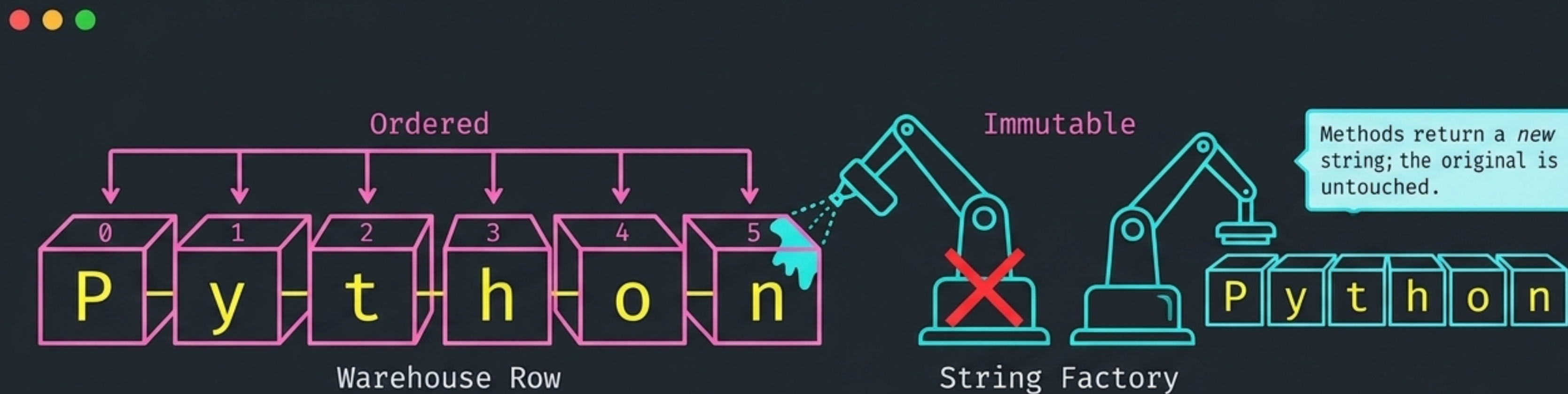
Python for Everybody: Mastering Strings

The ultimate visual guide to Python's
most versatile data structure.

Read-and-Share Cheat Sheet

Anatomy of a String

An ordered, immutable sequence of Unicode characters.



■ NOTE

Immutability means once a string is created, its characters cannot be altered. Methods that appear to "change" a string actually return a new string object.

Creating Strings

```
'...'  
"..."
```

Short strings

```
'''...'''  
"""..."""
```

Multi-line & Docstrings

Gluing Strings

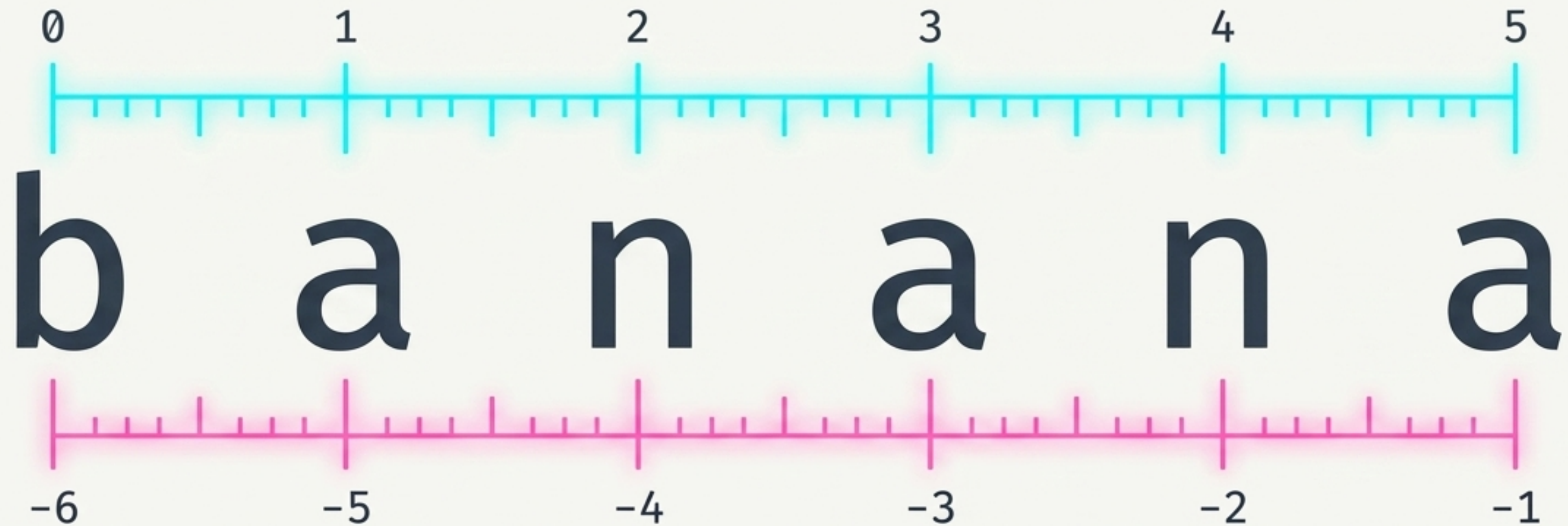
```
str1 = "Hello"  
str2 = 'there'  
bob = str1 + str2  
print(bob)      # Hellothere  
  
# Repetition operator  
line = '-' * 30  
print(line)     # -----
```

■ CAUTION



'3' + '4' = ~~7~~

CAUTION: Unlike numbers, '+' on strings does NOT add values — it glues them together. '3' + '4' gives '34', not 7.



```
# Positive indexing
print(fruit[0]) # First character
print(fruit[1]) # Second character
print(fruit[5]) # Last character (index = len-1)

# Negative indexing
print(fruit[-1]) # Last character
print(fruit[-2]) # Second from last
print(fruit[-6]) # Same as fruit[0]
```

The Pythonic Tail Grab

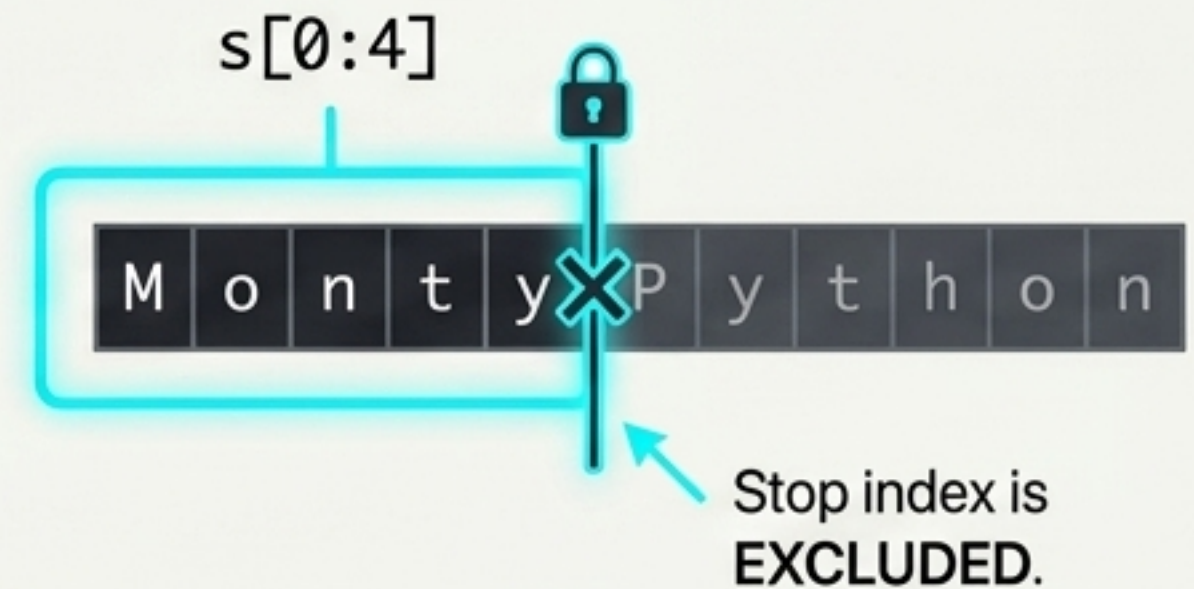
Negative index -1 always gives the last character regardless of string length. It is shorter, clearer, and works universally.

Slicing: The Substring Extractor

```
s = 'Monty Python'
```

```
print(s[0:4])  # Mont    → indices 0, 1, 2, 3 (4 excluded)
print(s[6:7])  # P        → just one character
print(s[:2])   # Mo       → from start up to index 2
print(s[8:])   # thon     → from index 8 to end
print(s[:])    # Monty Python → full copy
print(s[::-1]) # nohtyP ytnoM → reversed string
print(s[::-2]) # MnyPto   → every other character
```

The Slicing Window



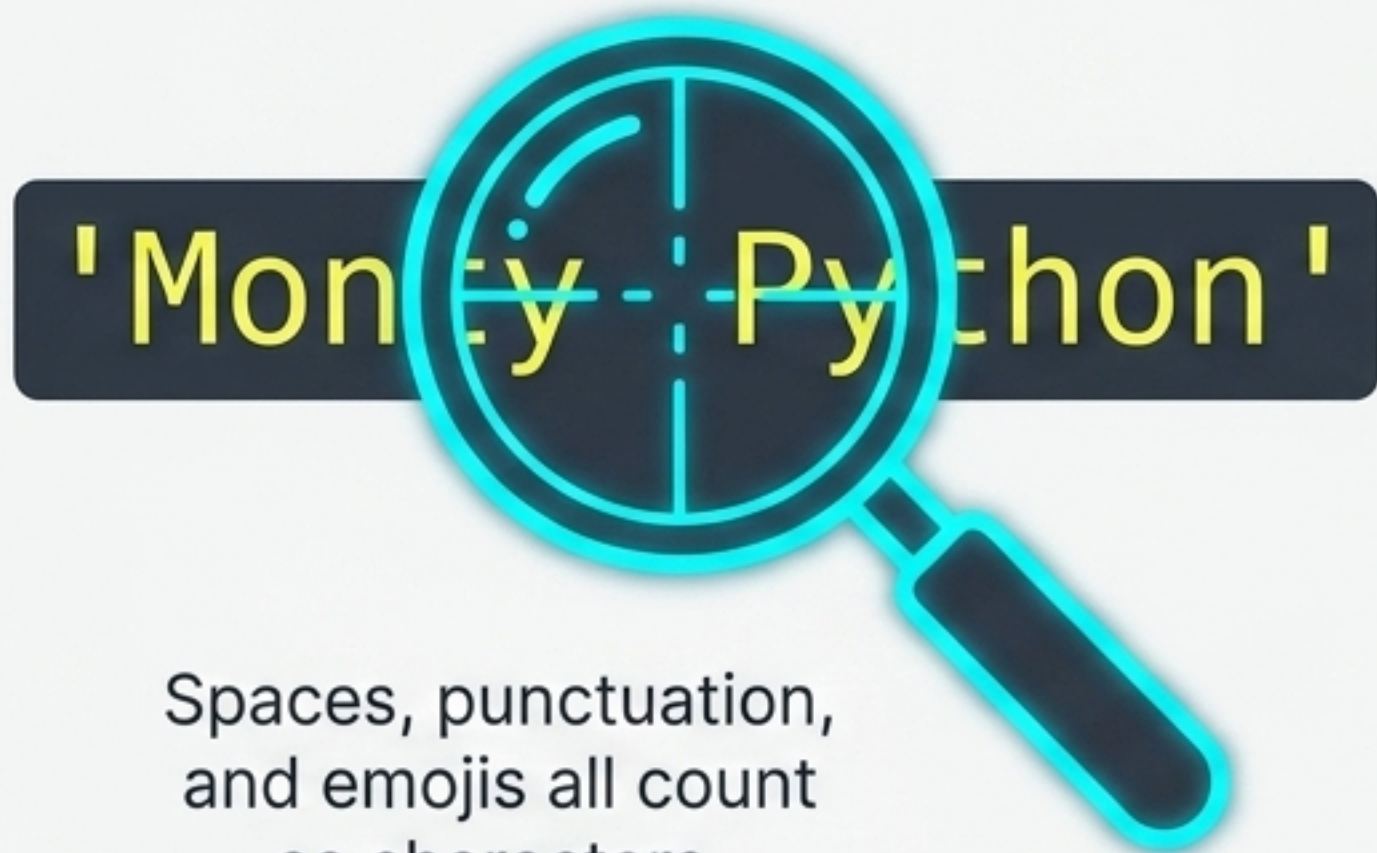
Monty Python

s[::-1]

The reverse idiom: flips the string backwards.

TIP If stop exceeds the string length, Python does **NOT** raise an error — it simply stops at the last character. So s[0:9999] safely returns the full string.

Concept



Spaces, punctuation, and emojis all count as characters.

Execution



`len()` operates in constant time. Python tracks the size internally, rather than counting characters one by one.

```
# Positive indexing
s = 'Monty Python'
print(len(s))      # 12 (space counts!)

# Common pattern: access last character safely
last = s[len(s) - 1]
print(last)        # n

# Even cleaner with negative index
print(s[-1])        # n
```


Looping Through Strings

Preferred & Pythonic for loop

```
fruit = 'banana'
for letter in fruit:
    print(letter)
```

- ✓ No index management
- ✓ Highly readable
- ✓ Iterates directly over characters.

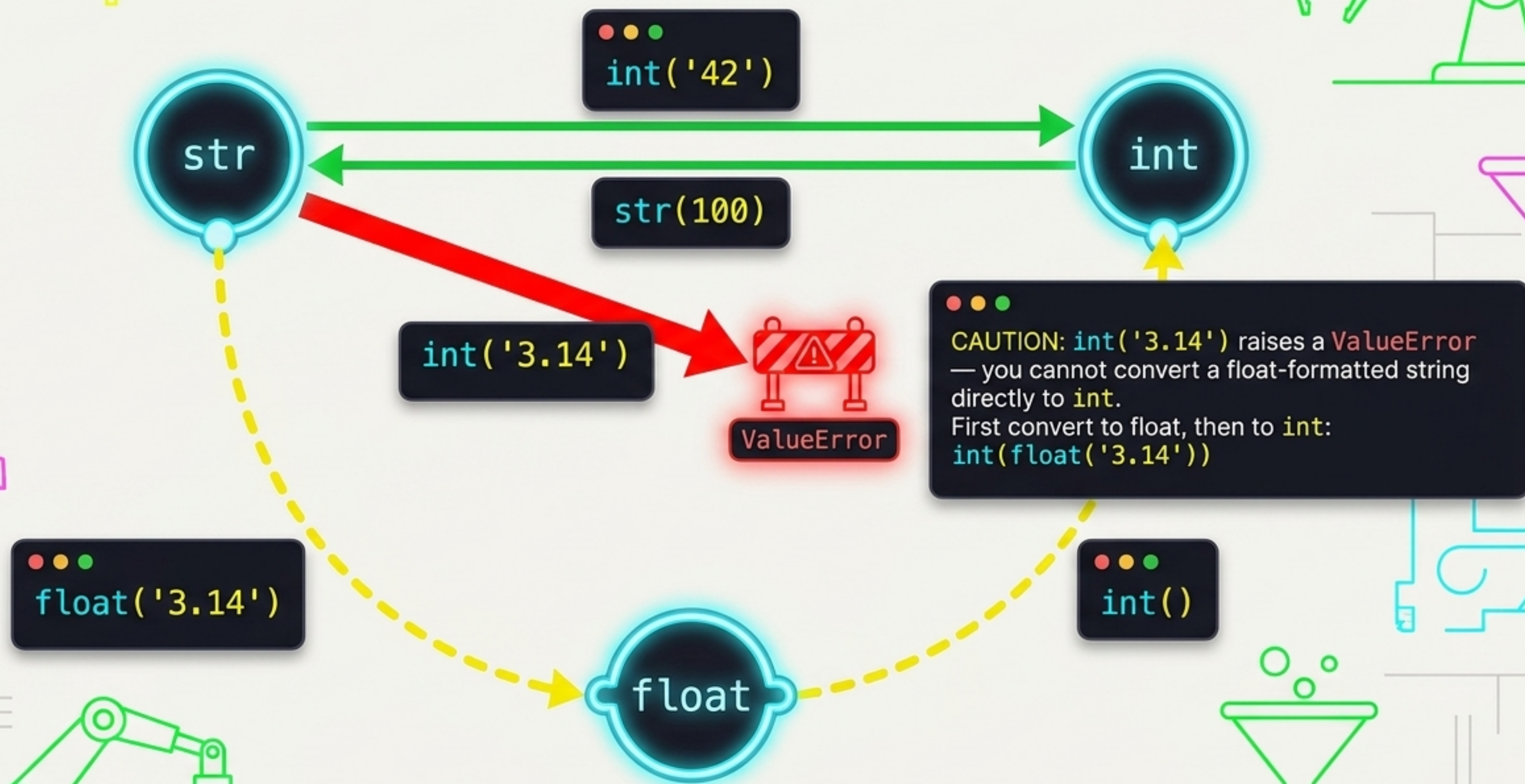
Verbose & Manual while loop

```
fruit = 'banana'
index = 0
while index < len(fruit):
    letter = fruit[index]
    print(letter)
    index = index + 1
```

- ⚠ Manual index tracking
- ⚠ Requires `index = index + 1`
- ⚠ High risk of infinite loops and off-by-one errors.

Synthesis Insight: Only use 'while' when explicit index manipulation (like skipping adjacent characters) is strictly necessary.

The Type Conversion Web



Membership Testing



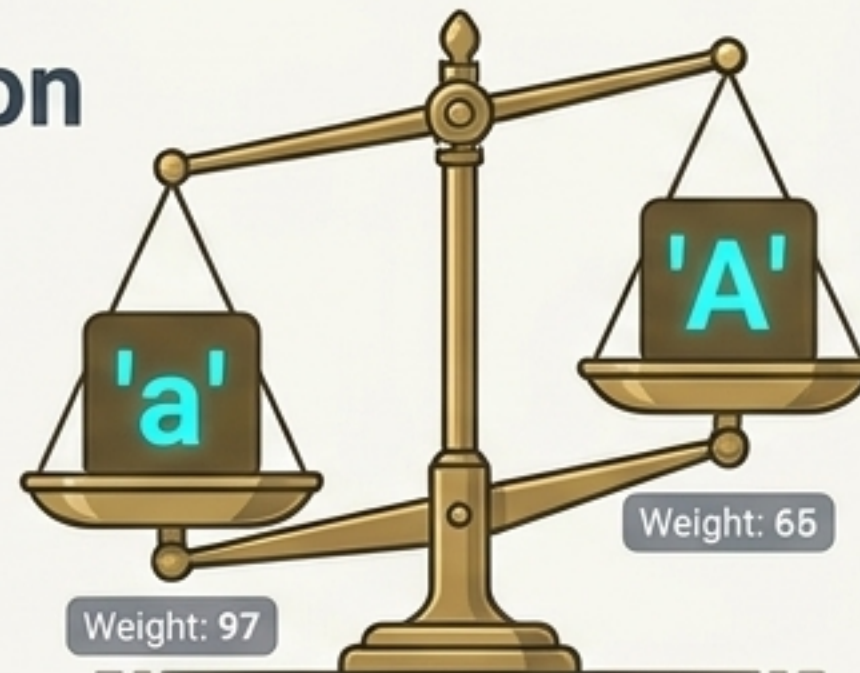
```
print('Python' in 'Monty Python') # True
```

```
print('Python' in 'Monty Python') # True
```

The fast, readable way to validate data without manual loops.

```
log = 'ERROR: connection timeout at 14:32'
if 'ERROR' in log:
    print('Alert! Error found in log.')
```

Lexicographical Comparison



```
word = 'cherry'
if word == 'banana':
    print('All right, bananas.')
elif word < 'banana':
    print(word, 'comes before banana alphabetically.')
elif word > 'banana':
    print(word, 'comes after banana alphabetically.')

# Output: cherry comes after banana alphabetically.
```

```
print(ord('a')) # 97
print(ord('A')) # 65 - uppercase is always smaller
```

Uppercase is always lighter. Therefore, 'a' > 'A' is mathematically True.

The String Methods Library

1. Case Normalisation

`.upper()`  `.lower()` `.capitalize()`

`str.upper()`

Converts all characters to uppercase.

`'hello'.upper()`

`'HELLO'`

2. Whitespace Stripping

`.strip()`  `.lstrip()` `.rstrip()`

`str.strip()`

Removes leading & trailing whitespace.

`' hi '.strip()`

`'hi'`

3. Search & Replace

`.find(sub)` `.replace(old, new)` 

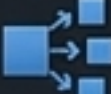
`str.replace(old, new)`

Replaces all occurrences of old with new.

`'banana'.replace('a', 'o')`

`'bonono'`

4. List Transitions

`.split(sep)`  `.join(iterable)`

`str.split(sep)`

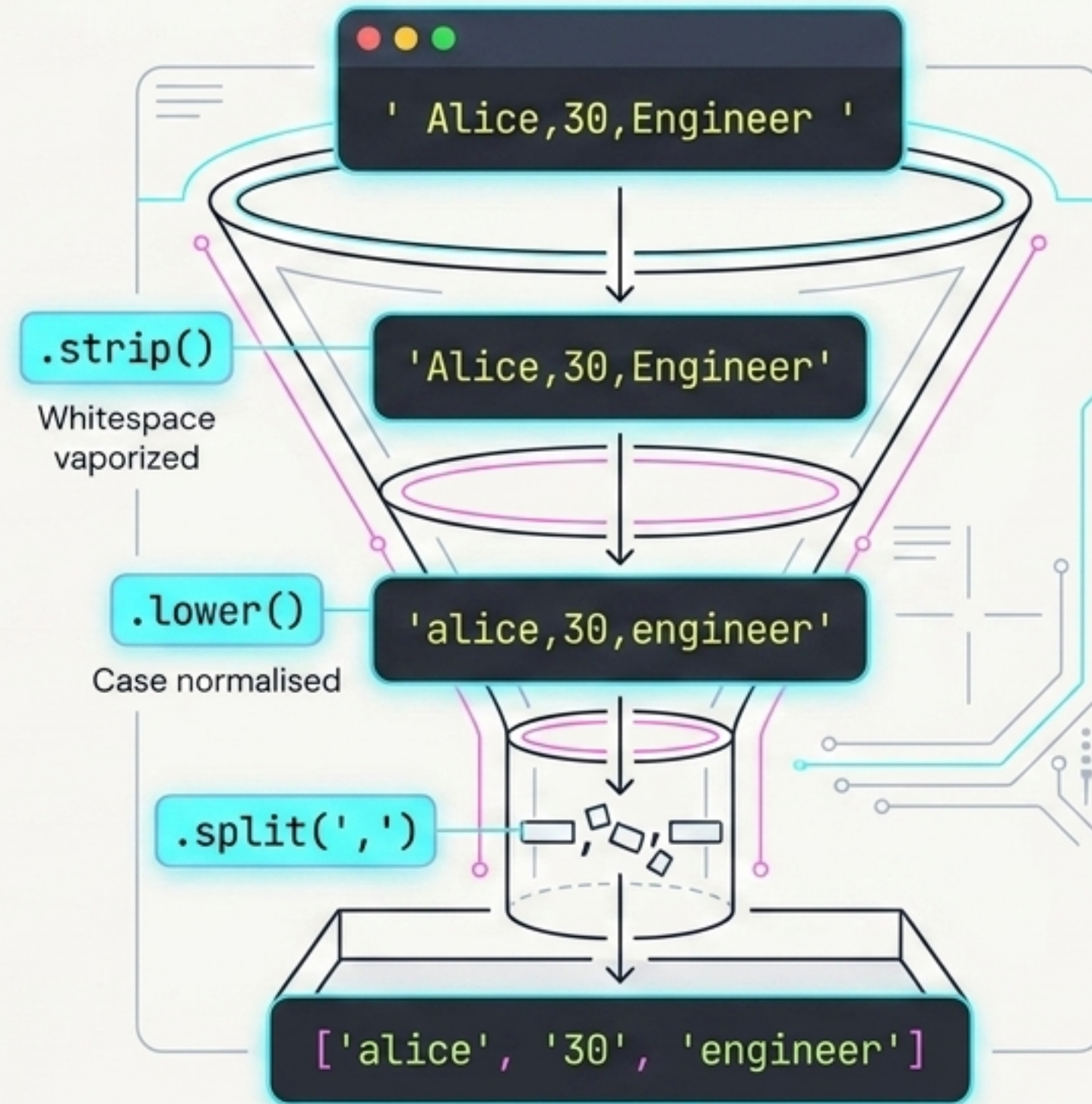
Splits string into a list of substrings.

`'a,b,c'.split(',')`

`['a', 'b', 'c']`

The Data Cleaning Trinity

String methods return **new** strings, allowing powerful, sequential chains.



Mini-Project: Real-world Log Parser

```
2024-01-15 14:32:07 \x1b[31mERROR\x1b[0m Connection timeout for user=admin
```

Raw data is useless. Our goal: Build a 4-step parsing machine to extract structured variables from this raw string.

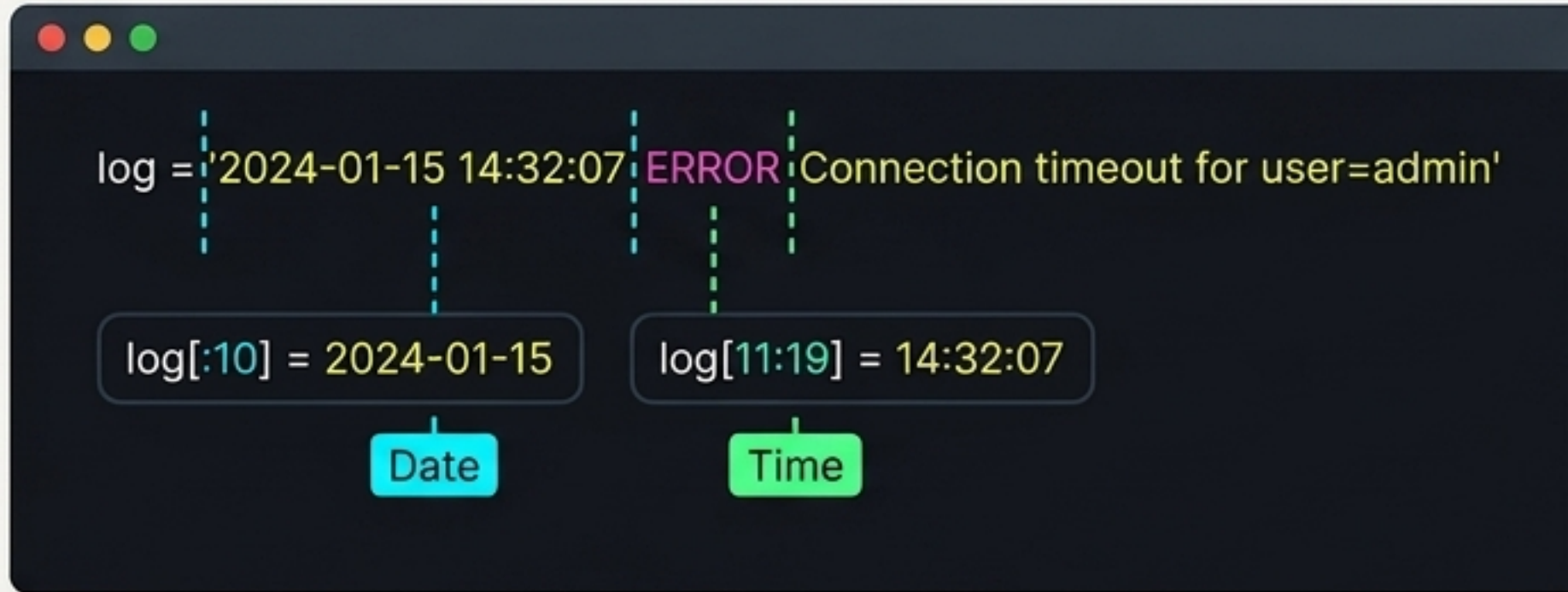
[Date]

[Time]

[Level]

[Message]

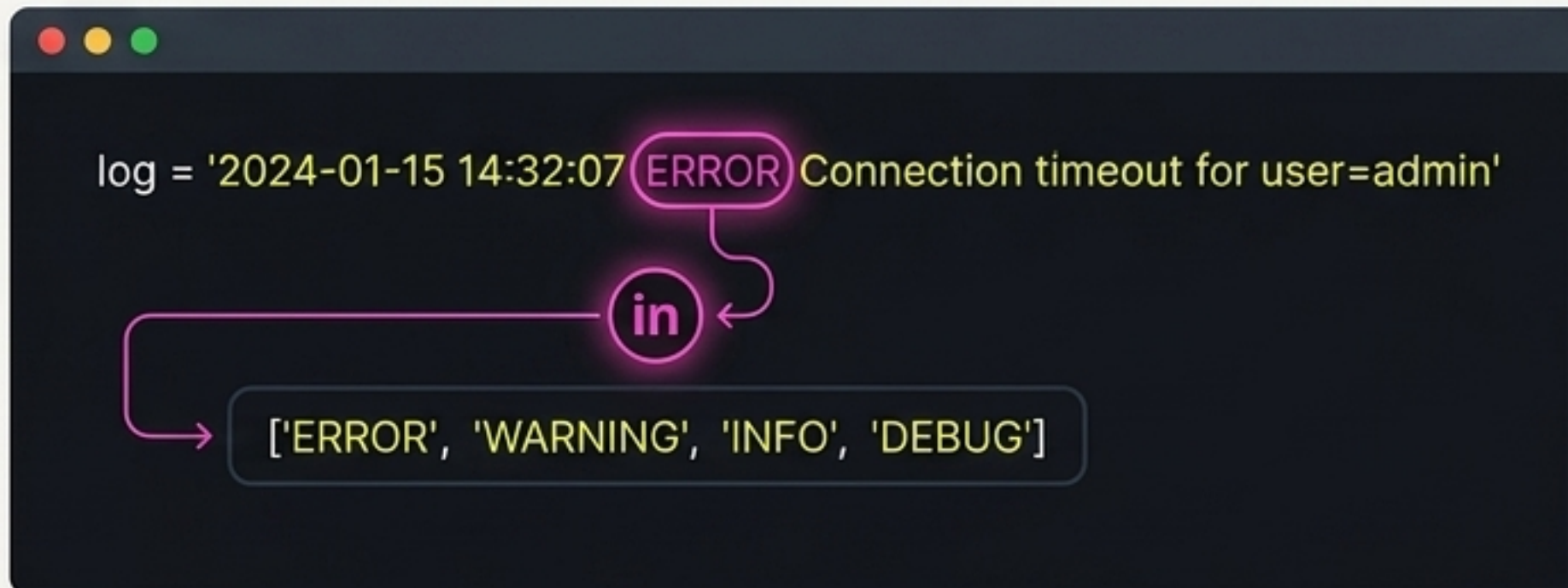
Step 1: The Slicer



Slicing is perfect for fixed-width data formats.

```
# 1. Extract date and time using slicing
date = log[:10]
time = log[11:19]
print(f'Date: {date}')
print(f'Time: {time}')
```

Step 2: The Membership Tester



Step 1 from the mini-project

```
# 2. Detect log level using 'in'
level = 'UNKNOWN'
for lvl in ['ERROR', 'WARNING', 'INFO', 'DEBUG']:
    if lvl in log:
        level = lvl
        break
print(f'Level: {level}')
```


Step 3: The Index Finder

```
idx = log.find(level)
```

```
log = '2024-01-15 14:32:07 ERROR Connection timeout for user=admin'
```

```
log[idx + len(level):].strip()
```

```
log = '2024-01-15 14:32:07 ERROR Connection timeout for user=admin'
```

"Connection timeout for user=admin"

Step 4: The Comparator

```
if level < 'WARNING':
```

E

W

Lexicographic Comparison:
'E' comes before 'W'

True



ALERT

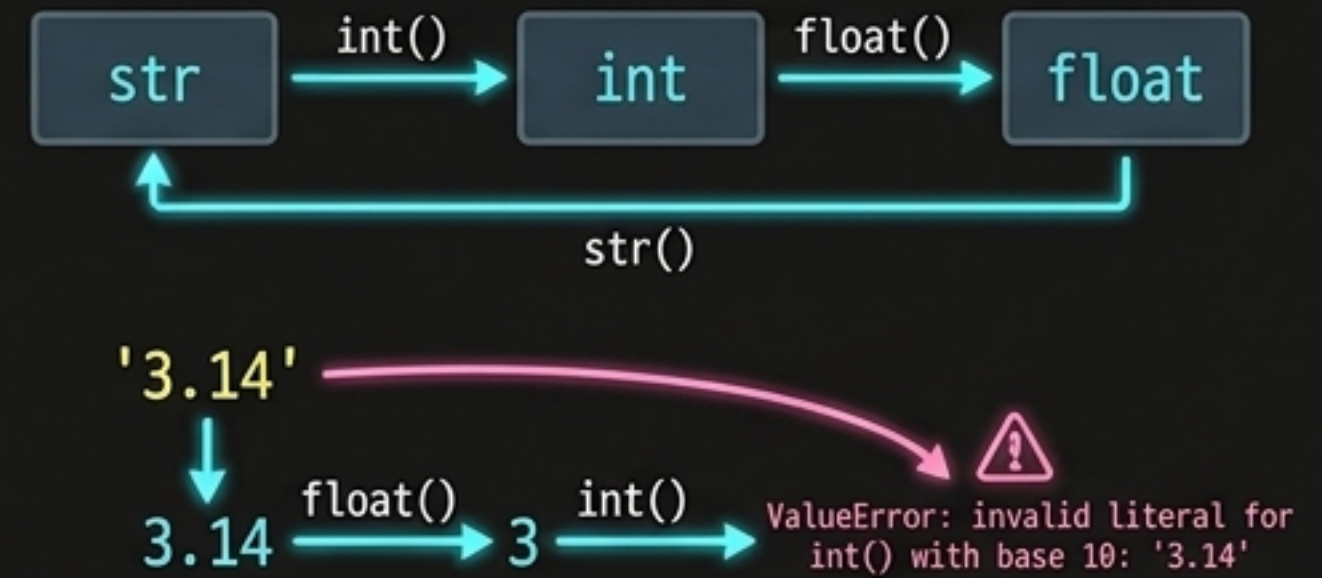
Critical issue detected!

Synthesis: String operations compose beautifully. Slice fixed widths, find() dynamic widths, in for testing, and strip() to clean.

Slicing & Indexing Map



Data Conversion



The Data Cleaning Equation

`raw_string → .strip() → .lower() → .split(',') → ['clean', 'list']`

Chains operations for efficient string processing.

Looping Matrix

Iterators

`for char in string:` ✓

Efficient, clean,
Pythonic iteration.

Manual Indices

`while index < len(s):` ⚠

Error-prone, verbose,
less readable.